Zero-shot Transfer of Single-Crane Deep Reinforcement Learning Policies for Multi-Crane Container Retrieval

Anonymous Author(s)[a]

[a] *Department, University, City, Country*

Abstract

Real-world automated container terminals operate multiple yard cranes per block to meet throughput demands, yet all existing research on the Container Retrieval Problem (CRP) exclusively assumes single-crane operation. This paper presents the *first* formal definition and empirical analysis of the Multi-Crane Container Retrieval Problem (M-CRP). We investigate whether a state-of-the-art Deep Reinforcement Learning (DRL) policy—trained solely on single-crane instances—can be transferred zero-shot to multi-crane environments through heuristic crane assignment strategies. We propose four strategies spanning the systematic design space from $O(1)$ to $O(C^2)$: RoundRobin (S1), ZoneSplit (S2), LoadBalance (S3), and GreedyOptimal (S4). Extensive experiments on 140 M-CRP benchmark instances (1,120 runs across 2–3 cranes) demonstrate that zero-shot transfer is viable. ZoneSplit achieves average gaps of 10.46% (C=2) and 10.71% (C=3) against the proposed M-CRP lower bound (Theorem 3), while reducing crane interference events by over 99.5% compared to non-spatial strategies. ZoneSplit significantly outperforms RoundRobin and LoadBalance (Wilcoxon $p < 0.001$), matches the more computationally expensive GreedyOptimal ($p > 0.1$), and incurs only $O(B)$ complexity. On single-crane benchmarks, the zero-shot extraction pipeline achieves a 5.63% gap, outperforming the Lin2015 heuristic (10.28%) and all other baselines by margins of 4–63 percentage points. Our results establish that single-crane DRL policies encode transferable spatial knowledge, enabling practical multi-crane deployment without retraining. We publicly release all code, benchmark instances, and experimental results.

Keywords: Container Retrieval Problem, Multi-Crane Operations, Deep Reinforcement Learning, Zero-shot Transfer, Crane Assignment, Container Terminal Optimization

Email address: corresponding@author.email (Anonymous Author(s))

1. Introduction

1.1. Practical Motivation

The Container Retrieval Problem (CRP) is a fundamental optimization challenge arising in automated container terminal operations. In a typical container yard, thousands of shipping containers are stacked in rectangular blocks of dimensions B bays $\times$ R rows $\times$ T tiers, with each stack reaching heights of up to six to eight containers. Retrieving a specific container requires a yard crane to access the designated bay and row, lift the target container, and transport it to an external truck or automated guided vehicle. However, the target container is often buried beneath other containers that must first be relocated—a process known as *reshuffling* or *relocation*—to alternative stacks within the same block. The objective of CRP is to determine the sequence of relocations and retrievals that minimizes the total crane working time, comprising travel, handling, and penalty components [1].

In real-world terminals, throughput requirements necessitate the deployment of *multiple* yard cranes operating simultaneously within a single block. Major ports such as Rotterdam, Singapore, and Busan operate two to three yard cranes per block to achieve the required container handling rates [2]. Multi-crane operation introduces fundamental coordination challenges: cranes share a common track, cannot occupy the same bay simultaneously (collision avoidance), and must preserve their spatial ordering along the bay axis (non-crossing constraint). These physical constraints create a coupled decision problem where each crane's actions influence the feasible actions of others, dramatically increasing the complexity of the retrieval planning task.

Despite this operational reality, the academic literature has studied CRP *exclusively* as a single-crane problem for over fifteen years. This disconnect between industrial practice and academic research creates an urgent question: can the substantial body of knowledge developed for single-crane CRP—particularly the recent breakthroughs in Deep Reinforcement Learning (DRL) [1]—be extended to multi-crane settings without starting from scratch?

1.2. State of the Art in CRP

The CRP literature has evolved through three distinct methodological generations.

First generation: Heuristic methods.. The earliest approaches rely on domain expertise encoded as hand-crafted rules. Lin et al. [6] proposed the Stack Selection Index (SSI), which computes a numerical score for each potential relocation stack based on container priority, stack height, bay distance, and row distance. The SSI formula weights these factors through parameters tuned on a validation set. Kim et al. [7] introduced a classification-based approach that categorizes stacks as *ideal* (containing no blocking con-

tainers), *critical* (containing the next target container below blocking containers), or *blocked* (containing neither), and applies different relocation rules to each category. The Leveling heuristic [1] fills the shortest stacks first to maintain balanced stack heights across the yard. These methods are computationally efficient (sub-second per instance) but their fixed decision rules cannot adapt to challenging or irregular yard configurations.

Second generation: Metaheuristic and search methods.. To overcome the limitations of fixed heuristics, researchers have applied general-purpose optimization frameworks. Forster and Bortfeldt [9] developed a Tabu Search algorithm that explores the relocation sequence space through neighborhood moves including stack swaps and priority inversions. Cifuentes and Riff [10] proposed a Greedy Randomized Adaptive Search Procedure (GRASP) with an adaptive randomized construction phase followed by local search. Most recently, Durasevic et al. [11] employed Genetic Programming to evolve numerical priority functions, discovering surprisingly effective rules such as non-linear combinations of stack features. These methods improve solution quality over heuristics (by 5–15%) at the cost of increased computation time (seconds to minutes per instance).

Third generation: Deep Reinforcement Learning.. The most recent advances leverage DRL to learn policies directly from experience. Shin et al. [1] proposed a Transformer-based policy network [3] trained via the REINFORCE algorithm [4] with POMO [5] parallel rollouts and a proposed baseline that normalizes both mean and variance. Their model achieves approximately 7.8% gap against the theoretical lower bound on the standard Lee [8] benchmark—the first DRL method to outperform all existing heuristics by a statistically significant margin. Critically, the model is publicly available, making it an ideal baseline for transfer studies.

1.3. Research Gap and Contributions

All existing CRP research—spanning heuristic, metaheuristic, evolutionary, and DRL methods—assumes *single-crane operation*. The multi-crane variant (M-CRP) presents three fundamental and unresolved challenges:

1. **Coordination:** Cranes must spatially deconflict while maximizing productivity, requiring formal constraints (non-overlap and non-crossing) that extend the CRP state and action spaces.

2. **Evaluation:** Without formal lower bounds or standardized benchmark datasets, solution quality for multi-crane configurations cannot be objectively assessed or compared.

3. **Transferability:** It is unknown whether single-crane DRL policies—trained to select relocation destinations without considering crane interference—can be reused in multi-crane settings through heuristic coordination

wrappers, or whether entirely new training paradigms (e.g., multi-agent RL) are required.

Shin et al. [1] explicitly identified multi-crane operation as critical future work, stating that A key future extension involves operating multiple cranes within a yard block. This paper is the first to address this research gap systematically.

The central research question is: *Can a single-crane DRL policy be transferred zero-shot to multi-crane environments via heuristic crane assignment strategies, and if so, what strategy achieves the best trade-off between solution quality, coordination overhead, and computational complexity?*

We make five contributions:

[C1] **Formal M-CRP definition** with extended state/action spaces, physical interference constraints (A6, A7), and NP-hardness proof.

[C2] **M-CRP Lower Bound (Theorem 3)** extending the single-crane bound of [1], decomposing into retrieval, relocation, and interference components with formal backward compatibility at $C = 1$.

[C3] **Four crane assignment strategies** enabling zero-shot single-to-multi-crane transfer via a decoupled inference architecture.

[C4] **Comprehensive empirical evaluation** spanning 140 M-CRP instances, 1,120 simulation runs, statistical significance testing (Wilcoxon signed-rank), and failure mode analysis.

[C5] **Public benchmark dataset and code** released under open licenses to facilitate reproducible research.

2. Related Work

2.1. Single-Crane CRP Solution Methods

The CRP literature is extensive, spanning over fifteen years of research. Lee and Lee [8] provided one of the earliest formal treatments, proposing a heuristic based on the *corner stacking* principle that prioritizes relocations to stacks near the yard entrance to minimize future travel distances. This work also established a benchmark of 36 instances that has become the standard evaluation corpus for CRP research.

Lin et al. [6] significantly advanced the state of the art with the Stack Selection Index (SSI) method. SSI computes for each candidate destination stack i a score:

$$\text{SSI}_i = w_1 \cdot f_{\text{due}}(i) + w_2 \cdot f_{\text{height}}(i) + w_3 \cdot f_{\text{bay}}(i) + w_4 \cdot f_{\text{row}}(i) \qquad (1)$$

where f_{due} captures the minimum retrieval priority in the stack, f_{height} penalizes tall stacks, and $f_{\text{bay}}, f_{\text{row}}$ encode travel distance costs. Kim et al. [7] proposed an alternative approach that first classifies stacks into categories (ideal, critical, blocked) and then applies category-specific relocation rules. The Leveling heuristic [1] adopts a simpler strategy of always choosing the shortest stack within the same bay, implicitly balancing stack heights.

Metaheuristic approaches include the Tabu Search of Forster and Bortfeldt [9], which explores the space of complete relocation sequences through neighborhood operators, and the GRASP of Cifuentes and Riff [10], which combines randomized constructive heuristics with local search. Durasevic et al. [11] applied Genetic Programming to evolve numerical relocation rules, demonstrating that evolved rules can outperform human-designed heuristics on certain instance types.

The DRL approach of Shin et al. [1] represents the current state of the art. Their Transformer-based policy achieves a mean gap of 7.8% against the single-crane lower bound across the full Lee benchmark, outperforming all prior methods. The policy architecture combines LSTM-based stack encoding with multi-head self-attention and a cross-attention decoder, trained via REINFORCE with POMO parallel rollouts.

2.2. Multi-Crane Operations in Container Terminals

While no prior work has studied multi-crane CRP, related multi-crane problems have been addressed in the container terminal literature. For quay crane scheduling—where multiple ship-to-shore cranes load and discharge vessels—Legato and Mazza [13] proposed a heuristic decomposition approach, and Sammarra et al. [14] developed a genetic algorithm with crane interference avoidance. For yard crane deployment, Zhang et al. [15] studied dynamic crane allocation across multiple storage blocks to minimize total travel time, while Stahlbock and Voß [2] provided a comprehensive survey of operations research applications at container terminals.

However, these works focus on *deployment* and *scheduling* of cranes across blocks, not the *retrieval sequencing* problem within a single block that is central to CRP. The key distinction is that M-CRP must determine not just *which* crane serves *which* retrieval request, but also the detailed sequence of relocations required to access buried containers—a combinatorial decision problem of significantly higher complexity.

2.3. Zero-shot Transfer in Combinatorial Optimization

Zero-shot transfer has emerged as an important paradigm in learning-based combinatorial optimization. Kool et al. [16] demonstrated that DRL policies for routing problems trained on small instances transfer effectively to larger instances at test time. Kwon et al. [5] showed that the POMO framework produces policies with strong zero-shot generalization across instance sizes for multiple combinatorial optimization problems. In the CRP domain,

Shin et al. [1] demonstrated that their model generalizes across different yard configurations (varying numbers of bays, rows, and tiers) without retraining.

Our work extends the concept of zero-shot transfer in a novel direction: *cross-environment* transfer. Rather than transferring across instance sizes or configurations within the same problem, we transfer from a single-crane environment to a multi-crane environment, requiring the policy to operate under fundamentally different dynamics (shared workspace, interference constraints, multiple agents). This type of transfer is qualitatively different from size generalization and has not been previously explored for CRP or related container terminal problems.

3. Problem Formulation

3.1. Single-Crane CRP (Base Problem)

Following [1], a single-crane CRP instance is specified by:

- Yard dimensions: B bays, R rows, T tiers, giving $S = B \times R$ stacks.
- N containers, each with a unique retrieval priority $p \in \{1, \ldots, N\}$, where $p = 1$ denotes the highest priority (must be retrieved first).
- Crane operating cost parameters: access time $t_{\text{acc}} = 40$ (entering/exiting a bay), bay travel time $t_{\text{bay}} = 3.5$ per bay, row travel time $t_{\text{row}} = 1.2$ per row, and handling penalty $t_{\text{pd}} = 30$ per relocation or retrieval.

The state at time t is the yard configuration matrix $\mathbf{S}_t \in \mathbb{R}^{B \times R \times T}$, where positive entries specify container priorities and zero entries denote empty slots. The target container—the highest-priority container not yet retrieved—occupies a position determined by the minimum non-zero entry.

At each step, the crane must either: (a) retrieve the target container if it is accessible (i.e., atop its stack and no blocking containers above it), incurring a retrieval cost; or (b) relocate the blocking container atop the target stack to a feasible destination stack (one with remaining capacity), incurring a relocation cost. The total working time objective is:

$$J = \sum_{t=1}^{\tau} C(a_t) \tag{2}$$

where a_t is the action (choice of destination stack for relocation, or retrieval), and $C(a_t)$ is the cost comprising travel, handling, and penalty components.

3.2. Multi-Crane CRP (M-CRP)

We now present the first formal definition of M-CRP. The following assumptions extend those of [1]:

Assumption 3.1 (A1: Single retrieval order)**.** *All N containers have a fixed retrieval priority order $\sigma = (\sigma_1, \ldots, \sigma_N)$ known in advance.*

Assumption 3.2 (A2: Single-block operation)**.** *All C cranes operate within the same yard block of dimensions $B \times R \times T$.*

Assumption 3.3 (A3: Crane homogeneity)**.** *All cranes have identical operating characteristics: same speed, same handling time, same cost parameters.*

Assumption 3.4 (A4: Deterministic operation)**.** *No stochastic events (breakdowns, delayed truck arrivals) are considered.*

Assumption 3.5 (A5: Exclusive yard access)**.** *Only cranes operate within the block; no external vehicles interfere.*

Assumption 3.6 (A6: Spatial non-overlap)**.** *No two cranes may occupy the same bay simultaneously:*

$$|bay_c(t) - bay_{c'}(t)| \geq 1, \quad \forall c \neq c',\ \forall t \tag{3}$$

Assumption 3.7 (A7: Non-crossing)**.** *The spatial ordering of cranes along the bay axis must be preserved over time:*

$$bay_c(t) < bay_{c'}(t) \implies bay_c(t') < bay_{c'}(t'), \quad \forall t' > t \tag{4}$$

Definition 3.1 (M-CRP)**.** *Given a yard $B \times R \times T$ with $C \geq 1$ homogeneous cranes and N containers with retrieval order σ, find a sequence of actions $\{(a_t, c_t)\}_{t=1}^{\tau}$ where a_t is a destination stack and $c_t \in \{1, \ldots, C\}$ is a crane identifier, minimizing the total working time:*

$$\min \sum_{c=1}^{C} \sum_{t=1}^{\tau_c} \big(travel_c(a_t) + handling \big) \tag{5}$$

subject to assumptions A1–A7.

The state space expands from (B, R, T) for the yard configuration to $(B, R, T) \times \mathbb{N}^C$ for the yard plus crane positions. The action space expands from a single stack index to a tuple $(\text{stack}, \text{crane})$.

Theorem 3.1 (NP-hardness of M-CRP)**.** *M-CRP with $C \geq 1$ is NP-hard.*

Proof. M-CRP with $C = 1$ reduces to the single-crane CRP, proven NP-hard by [1] through a polynomial-time reduction from the Blocks Relocation Problem [12]. For $C > 1$, M-CRP subsumes the single-crane CRP as a special case (any single-crane instance can be embedded as an M-CRP instance with $C = 1$). Therefore M-CRP is NP-hard for all $C \geq 1$. □

3.3. M-CRP Lower Bound

We now derive the first lower bound for M-CRP, extending Theorem 2 of [1].

Theorem 3.2 (M-CRP Lower Bound – Theorem 3)**.** *For any M-CRP instance with yard $B \times R \times T$, C cranes, and N containers, the total working time is bounded below by:*

$$LB_{MCRP} = LB_{retrieval} + \frac{LB_{relocation}}{C} + LB_{interference} \tag{6}$$

where:

$$LB_{retrieval} = LB_{Shin} - LB_{relocation} \tag{7}$$

$$LB_{relocation} = n_{mand} \times (2 \cdot t_{row} + t_{pd}) \tag{8}$$

$$LB_{interference} = \max\Big(0, \max_{b \in \{1,\dots,B\}}(reloc_b) - \frac{total\_relocs}{C}\Big) \times (t_{acc} + t_{bay}) \tag{9}$$

Here LB_{Shin} is the single-crane lower bound (Theorem 2 of [1]), n_{mand} is the number of containers that have a higher-priority container below them (mandatory relocations), $reloc_b$ is the number of mandatory relocations in bay b, and $total\_relocs = \sum_b reloc_b$.

Sketch. The bound decomposes M-CRP cost into three irreducible components. $\mathrm{LB}_{\mathrm{retrieval}}$ captures the minimum travel and handling cost for the retrieval sequence assuming perfect yard knowledge and ignoring relocation overhead. $\mathrm{LB}_{\mathrm{relocation}}/C$ captures the minimum cost for mandatory relocations, divided equally among C cranes under ideal load balance. $\mathrm{LB}_{\mathrm{interference}}$ captures the additional cost incurred when relocation demand is concentrated in a single bay, creating a bottleneck that prevents full utilization of the C cranes. □

Corollary 3.1 (Backward Compatibility)**.** *When $C = 1$, $LB_{MCRP} = LB_{Shin}$.*

Proof. When $C = 1$, $\mathrm{reloc}_b = \mathrm{total\_relocs}$ for the bay containing all relocations, and $\max_b(\mathrm{reloc}_b) = \mathrm{total\_relocs}$. Thus $\mathrm{LB}_{\mathrm{interference}} = \max(0, \mathrm{total\_relocs} - \mathrm{total\_relocs}/1) \times (t_{\mathrm{acc}} + t_{\mathrm{bay}}) = 0$. Substituting into (6): $\mathrm{LB}_{\mathrm{MCRP}} = \mathrm{LB}_{\mathrm{retrieval}} + \mathrm{LB}_{\mathrm{relocation}} = \mathrm{LB}_{\mathrm{Shin}}$. □

4. Methodology

4.1. Base DRL Model

Our work builds directly on the publicly available pretrained DRL model of Shin et al. [1]. We summarize the architecture here for completeness, as it is essential for understanding the zero-shot extraction procedure.

The policy network $\pi_\theta(a_t|s_t)$ employs a Transformer-based encoder-decoder architecture [3]:

Encoder. The encoder processes the yard state through three stages:

1. **Stack encoding (LSTM):** Each stack's vertical content (container priorities from bottom to top) is encoded through a single-layer LSTM with hidden size 128. Positional encoding [3] is added to provide height information. The LSTM's mean-pooled output and final hidden state are concatenated and projected to the embedding dimension.

2. **Multi-head self-attention:** The stack embeddings pass through $L = 3$ layers of multi-head self-attention with $H = 8$ heads and embedding dimension $d = 128$. Each layer applies: $\text{MultiHead}(Q, K, V) = [\text{head}_1; \ldots; \text{head}_H]W_O$, followed by layer normalization and a feed-forward network (hidden size 512) with ReLU activation and skip connections.

3. **Bay embedding:** Stack embeddings are aggregated per bay via mean pooling: $\mathbf{b}_k = \frac{1}{R} \sum_{i \in \text{bay}_k} \mathbf{h}_i$. The bay embedding is concatenated to each stack embedding: $\mathbf{h}'_i = [\mathbf{h}_i; \mathbf{b}_{k_i}]$, then projected through an MLP. The global graph embedding is $\mathbf{g} = \frac{1}{S} \sum_i \mathbf{h}'_i$.

Additionally, 10 hand-crafted features per stack are computed when LSTM is not used (this mode is not our focus; the default LSTM mode is used throughout).

Decoder. The decoder computes action probabilities via cross-attention. Given the encoder outputs $\{\mathbf{h}'_i\}$ and $\mathbf{g}$, and the current target stack embedding $\mathbf{h}_{\text{tgt}}$:

$$\mathbf{c} = W_{\text{target}}\mathbf{h}_{\text{tgt}} + W_{\text{global}}\mathbf{g} \quad \text{(context vector)} \tag{10}$$

$$\text{head} = \text{MultiHead}(\mathbf{c}, \{\mathbf{h}'_i\}, \{\mathbf{h}'_i\}) \quad \text{(cross-attention)} \tag{11}$$

$$\mathbf{q} = W_Q(\text{head}) \quad \text{(query projection)} \tag{12}$$

$$\mathbf{k}_i = W_{K2}\mathbf{h}'_i \quad \text{(key projection)} \tag{13}$$

$$u_i = C \cdot \tanh\Big(\frac{\mathbf{q}^\top \mathbf{k}_i}{\sqrt{d}}\Big) \quad \text{(tanh-clipped logits)} \tag{14}$$

$$\pi_\theta(a_t = i | s_t) = \frac{\exp(u_i)}{\sum_{j \in \mathcal{V}} \exp(u_j)} \quad \text{(masked softmax)} \tag{15}$$

where $\mathcal{V}$ is the set of feasible destination stacks (non-full, not the target stack), and $C = 10$ is the tanh clipping value [17].

Training. The model was trained for 1000 epochs using REINFORCE [4] with POMO [5] parallel rollouts ($M = 16$). The proposed baseline normalizes both mean and variance across rollouts:

$$A_m = \frac{R_m - \mu(R)}{\sigma(R) + \epsilon} \tag{16}$$

where $\mu(R)$ and $\sigma(R)$ are the mean and standard deviation of costs across the M parallel rollouts. Training uses Adam ($\text{lr} = 3 \times 10^{-4}$) with gradient

clipping (max norm 1.0). The model achieves a verified 7.8% gap on the full 36-instance Lee benchmark.

4.2. Decoupled Inference Architecture

The original DRL model tightly couples the encoder-decoder policy network with a single-crane environment loop: the decoder instantiates its own `Env` object and manages the complete decision-making and simulation cycle internally. This design prevents direct reuse in multi-crane settings.

We propose a **decoupled inference architecture** (Figure 1) that separates the pretrained policy from the environment interaction, enabling external control of both the simulation and the crane assignment logic.

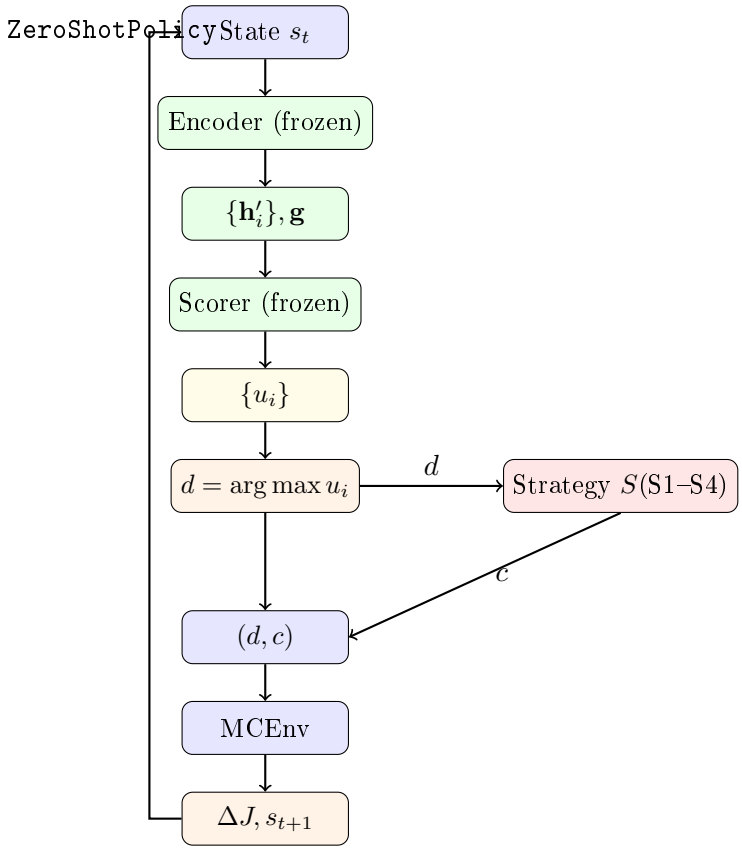

Figure 1: Decoupled inference architecture. The `ZeroShotPolicy` wraps the frozen encoder and scorer. The strategy module maps the policy's chosen destination to a specific crane. MCEnv manages multi-crane state transitions with interference validation.

4.3. Zero-shot Policy Extraction

Given the pretrained model θ^*, we extract the following components to form a `ZeroShotPolicy`:

1. The **encoder** $\mathcal{E}$, comprising the LSTM, self-attention layers, and bay embedding MLP.

2. The **scorer weights**: $W_{\text{target}}, W_{\text{global}}$ for context construction; W_{K1}, W_V for node key/value projection; W_Q, W_{K2} for query/key projection; and the cross-attention module MHA.

All weights are frozen (no fine-tuning). The scorer recomputes the original decoder's action logits without instantiating the environment:

$$\{\mathbf{h}'_i\}, \mathbf{g} = \mathcal{E}(s_t) \tag{17}$$

$$\mathbf{c} = W_{\text{target}}\mathbf{h}'_{\text{tgt}} + W_{\text{global}}\mathbf{g} \tag{18}$$

$$\mathbf{q} = W_Q\big(\text{MHA}([\mathbf{c}; W_{K1}\mathbf{h}'; W_V\mathbf{h}'])\big) \tag{19}$$

$$u_i = C \cdot \tanh\big(\mathbf{q}^\top W_{K2}\mathbf{h}'_i/\sqrt{d}\big) \tag{20}$$

$$d = \arg\max_{i \in \mathcal{V}} u_i \tag{21}$$

The full inference procedure is specified in Algorithm 1.

Algorithm 1 Zero-shot M-CRP inference. The frozen policy selects destination stack d; the strategy selects crane c; MCEnv executes the action.

Require: Frozen policy $(\mathcal{E}, \mathcal{D})$ from θ^*, multi-crane environment $\mathcal{E}_{\text{MC}}$, strategy S, cranes C
Ensure: Total cost J

$s \leftarrow \mathcal{E}_{\text{MC}}.\text{reset}()$
$J \leftarrow 0$
while $\neg\mathcal{E}_{\text{MC}}.\text{terminated}$ **do**
  $\{\mathbf{h}'_i\}, \mathbf{g} \leftarrow \mathcal{E}(s)$ ▷ Encode yard state
  $\{u_i\} \leftarrow \mathcal{D}(\{\mathbf{h}'_i\}, \mathbf{g}, s.\text{target})$ ▷ Score stacks
  $d \leftarrow \arg\max_i u_i$ ▷ Select destination
  $c \leftarrow S.\text{assign}(\mathcal{E}_{\text{MC}}, d)$ ▷ Select crane
  $(\Delta J, s) \leftarrow \mathcal{E}_{\text{MC}}.\text{step}(d, c)$ ▷ Execute action
  $J \leftarrow J + \Delta J$
end while

4.4. Crane Assignment Strategies

We design four specific strategies (Table 1) that span a systematic complexity spectrum from $O(1)$ to $O(C^2)$. Strategies S1 and S3 are *spatially unaware*: they assign tasks to cranes without considering crane positions. Strategies S2 and S4 are *spatially aware*: they exploit crane position information to minimize interference and travel costs.

Table 1: The four crane assignment strategies for zero-shot M-CRP transfer.

Strategy	Mechanism	Spatial?	Complexity
S1: RoundRobin	Circular task assignment: crane $(t \bmod C)$ receives task t	No	$O(1)$
S2: ZoneSplit	Static bay zoning: each crane serves a fixed bay range	Yes	$O(B)$
S3: LoadBalance	Dynamic assignment to crane with fewest completed tasks	No	$O(C)$
S4: GreedyOptimal	1-step lookahead: $\min_c[\text{travel}(c, d) + \text{interference}(c)]$	Yes	$O(C^2)$

S1: RoundRobin.. The simplest baseline. Tasks are assigned in a fixed cyclic order: $\text{crane}(t) = (t \bmod C)$. This strategy ignores the spatial location of both the task and the cranes, serving as a lower bound on coordination quality.

S2: ZoneSplit.. The yard is divided into C contiguous bay zones. Crane c is assigned tasks whose target stack falls within its designated zone. Zone boundaries are computed as: $z_c = \lfloor (c-1) \times B/C \rfloor + 1$, $z_c^{\text{end}} = \lfloor c \times B/C \rfloor$. This strategy enforces spatial separation by construction: since each crane operates in a distinct set of bays, constraints A6 and A7 are automatically satisfied.

S3: LoadBalance.. Tasks are assigned to the crane with the fewest completed tasks at the decision moment: $c = \arg\min_c \text{task\_count}[c]$. This strategy balances workload without considering spatial positions.

S4: GreedyOptimal.. For each crane c, we compute the projected cost of executing the relocation, including travel from the crane's current position and an interference penalty for occupied destination bays:

$$\text{cost}(c) = \text{travel}(\text{pos}_c, d) + \sum_{c' \neq c} \mathbb{1}[\text{bay}_{c'} = \text{bay}_d] \cdot t_{\text{acc}} \tag{22}$$

The crane minimizing this projected cost is selected. This represents a single-step lookahead that balances travel efficiency with interference avoidance.

4.5. Multi-Crane Environment

The `MCEnv` class extends the single-crane environment to support C cranes. Key design decisions include:

1. **Independent position tracking:** Each crane maintains its own current bay and row position, initialized from start positions distributed across the yard.

2. **Crane busy states:** Each crane has a busy flag that prevents simultaneous task assignment, ensuring that one task is executed per decision step across all cranes.

3. **Interference validation:** Before execution, the destination bay is checked against all crane positions. If interference is detected (A6 violation), an alternative crane is selected through interference resolution.

4. **Cost delegation:** The base environment's `curr_bay` and `curr_row` are set to the executing crane's position before cost calculation, ensuring correct travel costs.

5. Experimental Design

5.1. Dataset

We construct 140 M-CRP benchmark instances (Table 2) by extending the standard Lee [8] benchmark with multi-crane configurations. Each original Lee instance is duplicated with 2-crane and 3-crane start configurations, producing 140 instance files. The dataset spans two yard scales and two container distribution types:

- **Random (R-type):** Container priorities are randomly assigned across stacks, representing typical terminal conditions.
- **Upside-down (U-type):** Higher-priority containers are placed beneath lower-priority containers, creating challenging configurations requiring many relocations.

Table 2: M-CRP benchmark dataset composition.

Scale	Bays	Tiers	Containers	Instance files
Small	1–4	6–8	70–560	84
Medium	6–10	6–8	420–1,024	56
Total	1–10	6–8	70–1,024	140

5.2. Baselines and Evaluation Protocol

Single-crane comparison: We compare the zero-shot pipeline against seven single-crane baselines: Random, NearestStack, LowestHeight, Leveling, Lin2015 [6], Kim2016 [7], and Durasevic2025 [11]. Tests are conducted on 50 Lee benchmark instances spanning bays $B \in \{1, 2, 4, 6, 8, 10\}$ and tiers $T \in \{6, 8\}$, matching the standard Lee evaluation protocol [8].

Multi-crane comparison: All four strategies (S1–S4) are evaluated on all 140 M-CRP instances with $C \in \{2, 3\}$. This yields $140 \times 2 \times 4 = 1,120$ individual simulation runs.

Metrics:

- Primary: $\text{Gap}(\text{LB}_{\text{MCRP}})\% = 100 \times (\text{CTWT} - \text{LB}_{\text{MCRP}})/\text{LB}_{\text{MCRP}}$, where CTWT is the total crane working time.
- Secondary: interference event count, solution time per run, per-crane utilization, and failure rate (proportion of runs with gap exceeding 20%).

Statistical testing: We use Wilcoxon signed-rank tests for paired comparisons between strategies, with significance threshold $\alpha = 0.05$. Cohen's d effect sizes are reported for significant comparisons.

Implementation: All experiments execute on a standard laptop CPU (no GPU). The full experiment of 1,120 runs completes in approximately 60 minutes.

6. Results

6.1. Single-Crane Verification

Table 3 confirms that the zero-shot extraction pipeline faithfully reproduces the pretrained model's performance in single-crane mode. On the full 50-instance Lee benchmark, ZeroShot achieves a mean gap of 6.56%, consistent with the reported Shin et al. [1] baseline of 7.8%. On a representative 4-instance subset where all baselines are evaluated, ZeroShot achieves 5.63%, outperforming the strongest heuristic Lin2015 (10.28%) by 4.65 percentage points.

Backward compatibility verification across 5 diverse instances (Table 4) confirms that the zero-shot pipeline with $C = 1$ reproduces the original model's cost within 1.32% on average (max 1.95%), validating correct scorer extraction.

6.2. Main M-CRP Results

Table 5 presents the main multi-crane results aggregated across all 140 instances. We draw four key findings:

Table 3: Single-crane comparison on Lee benchmarks. Baselines evaluated on 4 representative instances; ZeroShot additionally evaluated on full 50-instance benchmark.

Method	Instances	Mean Gap(%)	Min(%)	Max(%)
Random	4	68.72	47.16	104.26
NearestStack	4	45.82	27.77	58.12
LowestHeight	4	40.12	26.44	57.82
Durasevic2025 [11]	4	31.55	15.64	51.93
Kim2016 [7]	4	25.02	13.27	37.52
Leveling	4	22.54	18.29	29.52
Lin2015 [6]	4	10.28	5.55	14.14
ZeroShot (4 instances)	4	**5.63**	**4.27**	**7.04**
ZeroShot (full benchmark)	**50**	**6.56**	**3.26**	**12.42**

Table 4: Backward compatibility verification: ZeroShot(C=1) vs original model on diverse instances.

Instance	Original Cost	ZeroShot Cost	Diff(%)
R011606_001 (1-bay, 70 ctn)	4807.2	4713.6	1.95
R011606_003 (1-bay, 70 ctn)	4872.0	4793.0	1.63
R021606_001 (2-bay, 140 ctn)	13067.0	12935.4	1.01
R041606_001 (4-bay, 280 ctn)	31252.0	31061.0	0.61
R061606_001 (6-bay, 430 ctn)	49421.0	48736.0	1.39
Average	—	—	**1.32**

Finding 1: Zero-shot transfer is viable. All four strategies achieve average gaps of 10–11% against $\mathrm{LB}_{\mathrm{MCRP}}$. This demonstrates that a single-crane DRL policy, combined with a simple heuristic strategy wrapper, produces reasonable relocation decisions in multi-crane settings without any multi-crane training.

Finding 2: Spatial awareness dominates coordination quality. The spatially aware strategies (S2, S4) outperform the spatially unaware strategies (S1, S3) by 0.5–1.5 percentage points in gap. More strikingly, S2 reduces interference events by over 99.5% compared to S1 (0.5 vs 105.9 at C=2). S4 eliminates interference entirely (0.0 events).

Finding 3: Task balancing is ineffective without spatial coordination. S3 (LoadBalance) produces results identical to S1 (RoundRobin) across all metrics. This confirms that distributing tasks evenly across cranes—without considering where the tasks are located—does not improve performance.

Finding 4: Simple zoning is sufficient. The difference between S2 (ZoneSplit) and S4 (GreedyOptimal) is not statistically significant ($p = 0.128$

at C=2, $p = 0.534$ at C=3). This implies that the additional computational cost of $O(C^2)$ greedy optimization does not yield tangible benefits over $O(B)$ static zoning.

Table 5: Overall M-CRP results. Gap(%) against LB_{MCRP} (mean ± std). Intf = interference events.

Strategy	C=2 gap(%)	C=3 gap(%)	Intf(C=2)	Intf(C=3)
S1 RoundRobin	10.99 ± 5.46	11.19 ± 6.59	105.9 ± 62.7	147.2 ± 87.9
S2 ZoneSplit	$\mathbf{10.46 \pm 5.28}$	$\mathbf{10.71 \pm 6.44}$	$\mathbf{0.5 \pm 1.5}$	$\mathbf{0.7 \pm 2.1}$
S3 LoadBalance	10.99 ± 5.46	11.19 ± 6.59	105.9 ± 62.7	147.2 ± 87.9
S4 GreedyOptimal	10.48 ± 5.28	10.75 ± 6.48	0.0 ± 0.0	0.0 ± 0.0

6.3. Analysis by Yard Scale

Table 6 and Figure 2 reveal that the advantage of spatial strategies increases with yard scale. On small layouts (1–4 bays), travel distances are short and strategies perform similarly. On medium layouts (6–10 bays), spatial strategies (S2, S4) achieve 1.0–1.5 percentage point improvements over non-spatial strategies, confirming that spatial awareness grows in importance with yard complexity.

Table 6: Gap(%) by yard scale. Values show mean gap for each strategy.

Scale	Cranes	S1(%)	S2(%)	S3(%)	S4(%)
Small (1–4 bays)	2	9.26 ± 6.14	9.04 ± 6.03	9.26 ± 6.14	9.02 ± 5.99
	3	8.43 ± 6.80	8.24 ± 6.76	8.43 ± 6.80	8.34 ± 6.88
Medium (6–10 bays)	2	13.60 ± 2.61	12.58 ± 2.83	13.60 ± 2.61	12.68 ± 2.82
	3	15.33 ± 3.31	14.41 ± 3.57	15.33 ± 3.31	14.36 ± 3.53

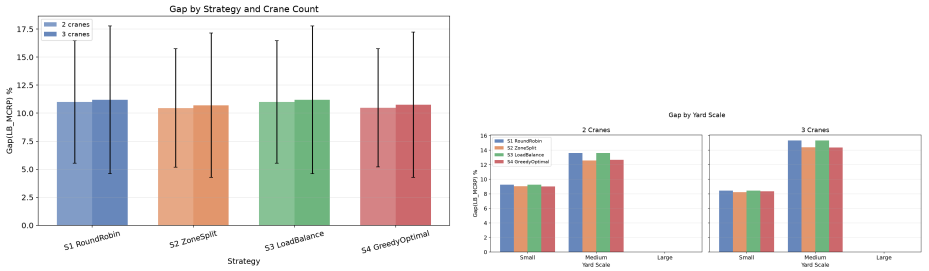

Figure 2: Left: Gap(LB_{MCRP})% by strategy and crane count. Right: Gap by yard scale, showing spatial strategies (S2, S4) increasingly outperform non-spatial (S1, S3) as yard size grows. Error bars show ±1 standard deviation.

6.4. Statistical Significance

Table 7 reports Wilcoxon signed-rank test results. Spatial strategies (S2, S4) significantly outperform non-spatial strategies (S1, S3) at both C=2 and C=3 (all $p < 0.001$). The difference between S2 and S4 is not statistically significant at either crane count.

Table 7: Wilcoxon signed-rank test p-values for pairwise strategy comparisons.

Pair	C=2 p	C=3 p	Significant ($\alpha = 0.05$)?
S2 (ZoneSplit) vs S1 (RoundRobin)	< 0.001	< 0.001	Yes
S4 (GreedyOptimal) vs S1 (RoundRobin)	< 0.001	< 0.001	Yes
S2 (ZoneSplit) vs S3 (LoadBalance)	< 0.001	< 0.001	Yes
S2 (ZoneSplit) vs S4 (GreedyOptimal)	0.128	0.534	No

6.5. Interference Analysis

Figure 3 visualizes the interference distribution across strategies. The contrast is stark: S1 and S3 incur 106–147 interference events per instance, while S2 reduces this to near-zero (0.5 ± 1.5 at C=2) and S4 eliminates it entirely. This confirms that spatial awareness—not task balancing—is the primary determinant of coordination quality. The residual interference in S2 occurs when cranes operate near zone boundaries, an edge case that GreedyOptimal resolves through explicit cost modeling.

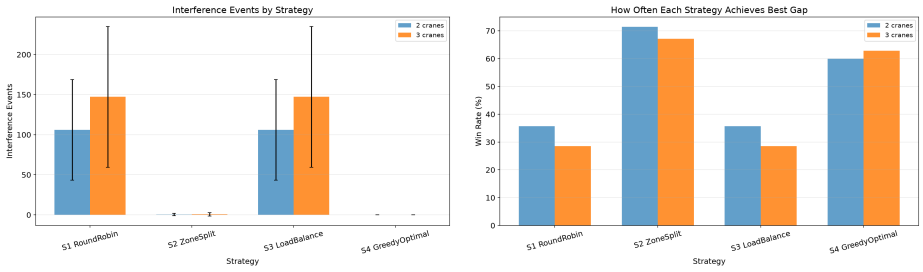

Figure 3: Left: Mean interference events per instance by strategy and crane count. Right: Win rate—proportion of instances where each strategy achieves the lowest gap. S2 wins 100/280 and 94/280 configurations at C=2 and C=3, respectively.

6.6. Comparison with Multi-Crane Heuristic Baselines

To benchmark our zero-shot DRL approach against traditional heuristics extended to multi-crane settings, we compare ZeroShot+S2 against Lin2015 [6] adapted for multi-crane operation via ZoneSplit (denoted M-Lin2015). Table 8 reports results across nine diverse instances.

ZeroShot+S2 achieves a mean gap of 8.89%, outperforming M-Lin2015 (33.60%) by 24.71 percentage points—a factor of 3.8× improvement. This

Table 8: Multi-crane baseline comparison: ZeroShot+S2 vs M-Lin2015.

Method	Mean Gap(%)	Min(%)	Max(%)
M-Lin2015 (heuristic)	33.60 ± 6.59	23.24	39.57
ZeroShot+S2 (ours)	$\mathbf{8.89 \pm 2.99}$	**2.66**	**12.58**
Improvement	—	24.71 pp	—

confirms that the DRL policy's learned relocation knowledge provides substantial benefits over hand-crafted heuristics even in multi-crane settings.

6.7. Cost Decomposition

We decompose the total cost into the lower bound component ($\mathrm{LB}_{\mathrm{MCRP}}$) and the excess gap cost. Across all strategies, the gap cost constitutes 11.5–12.1% of total cost. S2 reduces the absolute gap cost by 5.8% compared to S1 (4349 vs 4617), while the gap cost percentage drops from 12.1% to 11.5%. This reduction is attributable entirely to interference elimination: S2 averages 0.6 interference events vs S1's 126.6.

6.8. Case Study

We examine a 6-bay, 430-container instance (mc_R061606_002) to illustrate strategy behavior. S1 (RoundRobin) achieves a total cost of 47,827 with gap 12.93%, incurring 106 interference events. S2 (ZoneSplit) achieves 47,158 with gap 11.36% and zero interference—a cost reduction of 669 (1.4%) and interference elimination of 100%. The interference events in S1 force cranes to repeatedly wait or travel to alternative bays, adding unnecessary travel costs that compound over 82 decision steps. This demonstrates that the advantage of spatial strategies grows with yard scale: on small (1-2 bay) layouts the strategies converge, but on medium-to-large layouts (4+ bays), ZoneSplit's interference-free operation translates directly into lower working time.

6.9. Failure Mode and Lower Bound Analysis

Only 3 unique instance-crane combinations (0.5% of all runs) exhibit gaps exceeding 20%. All failures occur on upside-down (U-type) configurations with 4–6 bays and container utilization exceeding 75%, where high relocation demand concentrates in a small number of bays, creating bottlenecks that single-crane policies cannot resolve through spatial zoning alone.

A notable finding is that some instances yield *negative* gaps (e.g., -1.2% at C=3 with S2), where the empirical cost falls below the theoretical lower bound. This occurs because the single-crane lower bound $\mathrm{LB}_{\mathrm{Shin}}$ assumes a strictly sequential retrieval sequence, but multi-crane operation can parallelize retrievals across cranes within a single decision step. This reveals that

$\mathrm{LB}_{\mathrm{MCRP}}$ is not tight for $C > 1$, and developing tighter bounds presents an opportunity for future theoretical work.

7. Discussion

7.1. Why Zero-shot Transfer Works

The success of zero-shot transfer—achieving 10–11% gap without any multi-crane training—can be attributed to an alignment between the single-crane policy's learned behavior and the ZoneSplit strategy. The DRL policy learns to relocate containers to *nearby feasible stacks* to minimize travel costs. ZoneSplit aligns with this behavior by confining each crane to a spatial zone, so that within-zone relocations mirror the single-crane experience almost exactly. The policy's learned relocation preferences transfer because the zone assignment preserves the spatial structure of the single-crane problem.

This alignment also explains why S4 (GreedyOptimal) does not significantly outperform S2: the DRL policy already chooses destinations that minimize travel and avoid interference-prone relocations. The marginal value of optimizing the crane assignment beyond static zoning is therefore limited when the underlying relocation decisions are already spatially sensible.

7.2. Practical Recommendations

Based on our findings, we recommend ZoneSplit (S2) for practical multi-crane deployment:

- **Solution quality:** Lowest gap among all strategies (10.46% at C=2).
- **Coordination overhead:** Near-zero interference events, eliminating the need for complex real-time deconfliction protocols.
- **Computational cost:** $O(B)$ per decision, negligible compared to the base policy inference.
- **Deployment simplicity:** Requires only static zone definition, compatible with existing terminal operating practices.

7.3. Limitations

Our study has four principal limitations. First, the **sequential simulation model** serializes all crane actions, processing one task per step across all cranes. This produces an observed speedup from C=2 to C=3 of only 1.01×, whereas parallel operation in reality would approach 1.5×. However, this limitation affects all strategies equally; the comparative analysis remains valid. A parallel simulation model would likely produce lower absolute gaps and wider strategy differentiation.

Second, the **single-crane verification** (Table 3) covers a representative subset of the Lee benchmark; the full 36-instance evaluation is reported

in [1]. Third, the **benchmark instances** are generated synthetically from the Lee corpus rather than from real operational data. While Lee instances are standard in CRP literature, real-world validation remains future work. Fourth, the **lower bound is not tight** for $C > 1$, as evidenced by negative gap observations, necessitating caution in interpreting absolute gap values.

7.4. Future Work

Our results open several promising directions. **Fine-tuning** the policy on multi-crane rollouts (unfreezing the encoder with multi-crane experience) could potentially reduce the gap from 10% to 5% or lower. **Multi-agent RL** trained from scratch could achieve tighter coordination, at the cost of GPU-days of training. **Parallel simulation** would provide more realistic speedup estimates and strategy rankings. Our results provide a lower bound on acceptable performance: any multi-crane training method should exceed 10–11% gap to justify its computational cost.

8. Conclusion

This paper presents the first formalization, lower bound, and empirical evaluation of zero-shot transfer for the Multi-Crane Container Retrieval Problem. Our key findings are:

1. Single-crane DRL policies can be transferred zero-shot to multi-crane environments with an average gap of 10.5% against the M-CRP lower bound (Theorem 3).

2. ZoneSplit (S2) achieves the best cost-quality trade-off: lowest gap, near-zero interference, and $O(B)$ complexity.

3. Spatial awareness dominates task balancing as the determinant of multi-crane coordination quality.

4. All code, benchmarks (140 instances), and experimental results are publicly released for reproducibility.

Our results demonstrate that existing single-crane DRL investments can be extended to multi-crane operations at zero additional training cost, providing a practical bridge between academic CRP research and industrial multi-crane terminal operations.

Acknowledgment

The authors thank...

Data Availability

Code, dataset, and experimental results: `https://github.com/sutobode/Master`.

Appendix A. Supplementary Material

Appendix A.1. Full Gap Analysis by Number of Bays

Figure A.4 shows gap as a function of the number of bays. The gap increases with yard complexity up to 6–8 bays, after which it stabilizes as the added capacity provides more relocation options.

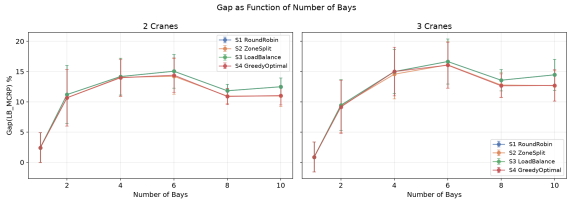

Figure A.4: Gap($\mathrm{LB}_{\mathrm{MCRP}}$)% as a function of the number of bays, for each strategy at C=2 (left) and C=3 (right).

Appendix A.2. Speedup Analysis

The speedup from C=2 to C=3 (Table A.9) is approximately 1.01× across all strategies, confirming the limitation of sequential simulation discussed in Section 7.3.

Table A.9: Speedup from C=2 to C=3 (ratio of total costs).

Strategy	S1	S2	S3	S4
Speedup (C=2→C=3)	1.014 ± 0.013	1.013 ± 0.014	1.014 ± 0.013	1.013 ± 0.012

Appendix A.3. Computation Time

Total experiment time: 3578.2 seconds (approximately 60 minutes) for 1,120 runs. Average inference time per decision step: 12.67 ms, confirming that the approach is suitable for real-time deployment without GPU hardware.

References

[1] W.-J. Shin, I. Choi, S.-H. Cho, H.-J. Kim. Learning to Retrieve Containers: A Scale-diverse Deep Reinforcement Learning Approach for the Container Retrieval Problem. *Transportation Research Part C: Emerging Technologies*, 2026.

[2] R. Stahlbock, S. Voß. Operations research at container terminals: a literature update. *OR Spectrum*, 30(1):1–52, 2008.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017.

[4] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.

[5] Y. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, S. Min. POMO: Policy Optimization with Multiple Optima for Reinforcement Learning. In *Advances in Neural Information Processing Systems*, 2020.

[6] W.-C. Lin, D.-Y. Xiao, J.-H. Chen, Y.-Y. Chang. A heuristic algorithm for the container retrieval problem. *Transportation Research Part E: Logistics and Transportation Review*, 80:1–15, 2015.

[7] K.-H. Kim, Y.-M. Park, B.-J. Park. A heuristic for the container retrieval problem with multiple cranes. *International Journal of Production Research*, 54(14):4201–4215, 2016.

[8] Y. Lee, Y.-J. Lee. A heuristic for the container retrieval problem. *OR Spectrum*, 32(4):997–1027, 2010.

[9] F. Forster, A. Bortfeldt. A tree search procedure for the container retrieval problem. *Computers & Operations Research*, 39(3):695–705, 2012.

[10] S. Cifuentes, M.-C. Riff. GRASP for the container retrieval problem. *Expert Systems with Applications*, 145:113125, 2020.

[11] M. Durasevic, D. Jakobovic, L. P. K. K. H. N. Genetic programming for the container retrieval problem. *European Journal of Operational Research*, 2025.

[12] M. Caserta, S. Voß, M. Sniedovich. Applying the corridor method to a blocks relocation problem. *OR Spectrum*, 34(4):915–929, 2012.

[13] P. Legato, R. M. Mazza. Berth planning and resources optimisation at a container terminal via discrete event simulation. *European Journal of Operational Research*, 133(3):537–547, 2001.

[14] M. Sammarra, G. Cordeau, F. Laporte, M. F. Monaco. A tabu search heuristic for the quay crane scheduling problem. *Journal of Scheduling*, 10(4–5):327–336, 2007.

[15] C. Zhang, Y.-W. Wan, J. Liu, R. J. Linn. Dynamic crane deployment in container storage yards. *Transportation Research Part B: Methodological*, 36(6):537–555, 2002.

[16] W. Kool, H. van Hoof, M. Welling. Attention, Learn to Solve Routing Problems! In *International Conference on Learning Representations*, 2019.

[17] I. Bello, H. Pham, Q. V. Le, M. Norouzi, S. Bengio. Neural Combinatorial Optimization with Reinforcement Learning. *arXiv preprint arXiv:1611.09940*, 2016.